

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting
(BL 3)

Assessment Tool Weightage: 10%

QUESTIONS: 10

Annexure A

Total Marks:20

MOCK INTERVIEW QUESTIONS

Code of Conduct:

I S. Dharani Teja (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Hospital Patient Data

Answer:

I would use Pandas for handling the CSV files and NumPy for numerical operations.

1. Read the CSV file using `pd.read_csv()`.
2. Check missing values using `isnull().sum()`.
3. Fill or remove missing values using `fillna()` or `dropna()`.
4. Remove duplicate records using `drop_duplicates()`.
5. Convert inconsistent units into a common unit.
6. Use Pandas indexing with `loc` and `iloc` to access required records.
7. Filter invalid or unwanted patient records using conditions.
8. Use `groupby()` and aggregation functions such as `mean()`, `sum()`, and `count()` for reports.
9. Handle unexpected values using validation rules and exception handling.
10. Save the cleaned dataset using `to_csv()`.

Example:

```
import pandas as pd
df = pd.read_csv("patients.csv")
df = df.drop_duplicates()
df["Age"] = df["Age"].fillna(df["Age"].mean())
df = df[df["Age"] > 0]
summary = df.groupby("Department")["Age"].mean()
df.to_csv("cleaned_patients.csv", index=False)
print(summary)
```

2. Financial Transaction Data

Answer:

For millions of transaction records, I would use NumPy arrays for fast numerical calculations and Pandas DataFrames for structured data analysis.

1. Read the transaction data using Pandas.
2. Select only required columns to reduce memory usage.
3. Convert numerical data into appropriate NumPy/Pandas data types.
4. Use Boolean filtering to identify unusual transactions.
5. Use `groupby()` for customer-wise or category-wise analysis.
6. Calculate statistics such as mean, median, standard deviation, minimum, and maximum.
7. Use indexes on frequently searched columns to improve access.
8. Avoid unnecessary loops and use vectorized NumPy/Pandas operations.
9. Check memory usage using `memory_usage()`.
10. Validate results and profile slow operations to identify performance bottlenecks.

Example:

```
import pandas as pd
```

```
import numpy as np
```

```
df = pd.read_csv("transactions.csv")
```

```
# Filter unusual transactions
```

```
unusual = df[df["Amount"] > 100000]
```

```
# Group transactions
```

```
summary = df.groupby("Customer_ID")["Amount"].agg(  
    ["sum", "mean", "max"]  
)
```

```
print(unusual)
print(summary)
```

This approach reduces unnecessary computation and makes analysis of large datasets more efficient.

3. Combining Department Datasets

Answer:

I would use Pandas to read and combine the sales, marketing, and customer-support datasets.

1. Read each file using the appropriate Pandas function such as `read_csv()` or `read_excel()`.
2. Check the column names and data types.
3. Standardize common column names.
4. Check for missing and duplicate keys.
5. Use `merge()` to combine datasets based on a common key such as `Customer_ID`.
6. Use `concat()` when datasets have similar structures.
7. Reset or maintain a consistent index using `reset_index()`.
8. Check the merged DataFrame for missing values and duplicate records.
9. Resolve conflicting values using predefined rules.
10. Sort the final dataset using `sort_values()`.

Example:

```
import pandas as pd
```

```
sales = pd.read_csv("sales.csv")
```

```
marketing = pd.read_csv("marketing.csv")
```

```
support = pd.read_csv("support.csv")

df = sales.merge(marketing, on="Customer_ID", how="left")
df = df.merge(support, on="Customer_ID", how="left")

df = df.drop_duplicates()
df = df.sort_values("Customer_ID")

print(df)
```

The merged dataset should be validated by checking row counts, missing keys, duplicate keys, and important columns.

4. Automated Daily Data Pipeline

Answer:

The system can be designed as an automated ETL pipeline using NumPy, Pandas, and Matplotlib.

Workflow:

Read → Clean → Filter → Aggregate → Sort → Visualize → Save

1. Read the daily data file using Pandas.
2. Check whether the file exists and is readable.
3. Validate required columns.
4. Clean missing and duplicate values.
5. Filter relevant records.
6. Calculate statistics using aggregation functions.
7. Sort the results for dashboard display.
8. Generate charts using Matplotlib.
9. Save the processed data and reports.

10. Use exception handling for corrupted files and incorrect formats.

Example:

```
import pandas as pd
import matplotlib.pyplot as plt

try:
    df = pd.read_csv("daily_data.csv")
    required = ["Date", "Sales"]
    if not all(col in df.columns for col in required):
        raise ValueError("Required columns are missing")
    df = df.drop_duplicates()
    df["Sales"] = df["Sales"].fillna(0)
    df = df[df["Sales"] > 0]
    summary = df.groupby("Date")["Sales"].sum()
    summary = summary.sort_values(ascending=False)
    summary.to_csv("summary.csv")
    summary.plot(kind="bar")
    plt.title("Daily Sales")
    plt.xlabel("Date")
    plt.ylabel("Sales")
    plt.show()
except Exception as e:
    print("Error:", e)
```

This workflow automatically processes daily files and produces sorted summaries and visualizations while handling common errors.

5. E-Commerce Clickstream Data

Answer:

I would use Pandas to clean, structure, and prepare the clickstream data for machine learning.

1. Load the clickstream data using `read_csv()`.
2. Check missing values using `isnull()`.
3. Remove duplicate records.
4. Convert dates and timestamps into a standard format.
5. Standardize categorical values such as device type and browser.
6. Filter invalid records.
7. Extract useful features such as:
 - Session duration
 - Number of clicks
 - Pages visited
 - Time of visit
 - Device type
8. Set an appropriate index such as `User_ID` or `timestamp`.
9. Encode categorical variables for machine learning.
10. Save the final cleaned dataset for downstream ML tasks.

Example:

```
import pandas as pd
df = pd.read_csv("clickstream.csv")
df = df.drop_duplicates()
df["Timestamp"] = pd.to_datetime(
    df["Timestamp"],
    errors="coerce"
)
df["Duration"] = df["End_Time"] - df["Start_Time"]
df = df.dropna(subset=["User_ID", "Timestamp"])
```

```
df = df[df["Clicks"] >= 0]
df = df.sort_values("Timestamp")
print(df.head())
```

The cleaned and structured DataFrame can then be given to a machine-learning model for customer behavior prediction and analysis.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	25	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand